

LAB1:

```
1  --Lab1
2  EXPLAIN (ANALYZE , BUFFERS )
3  SELECT c . first_name , c . last_name , r . rental_date
4  FROM customer c
5  JOIN rental r ON c . customer_id = r . customer_id
6  WHERE c . customer_id = 1 ;
```

Data Output Messages Notifications

Showing rows: 1 to 13 Page No: 1 of 1

	QUERY PLAN
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.104..1.315 rows=32 loops=1)
2	Buffers: shared hit=151 read=2
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.084..0.085 rows=1 loops=1)
4	Index Cond: (customer_id = 1)
5	Buffers: shared hit=1 read=2
6	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.017..1.224 rows=32 loops=1)
7	Filter: (customer_id = 1)
8	Rows Removed by Filter: 16012
9	Buffers: shared hit=150
10	Planning:
11	Buffers: shared hit=140 dirtied=4
12	Planning Time: 4.344 ms
13	Execution Time: 1.336 ms

Outer (top) relation: customer c

Inner (bottom) relation: rental r

Rows returned by customer: Exactly **1 row**

Why Hash Join is "too expensive": A Hash Join requires building an in-memory hash table for the inner relation and hashing keys before probing. Because the outer relation yields only a single row, running a one-time loop against the inner table has virtually zero setup overhead. The startup cost and memory allocation needed to construct a hash table far exceed the minimal cost of a simple Nested Loop iteration.

Why loops=1 on the Inner Table: Because the outer scan on customer returned only **1 row**, PostgreSQL executed the inner scan on rental exactly once (loops=1).

LAB2:

```
9  --Lab2
10 EXPLAIN (ANALYZE , BUFFERS )
11 SELECT f.title , l.name as language
12 FROM film f
13 JOIN language l ON f.language_id = l.language_id ;
```

Data Output	Messages	Notifications
Showing rows: 1 to 14		
Page No: 1 of 1		
SQL		
QUERY PLAN		
text		
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99) (actual time=0.048..0.617 rows=1000 loops=1)
2	Hash Cond	(f.language_id = l.language_id)
3	Buffers: shared hit=89	
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17) (actual time=0.012..0.409 rows=1000 loops=1)
5	Buffers: shared hit=88	
6	-> Hash	(cost=1.06..1.06 rows=6 width=88) (actual time=0.019..0.020 rows=6 loops=1)
7	Buckets: 1024 Batches: 1	Memory Usage: 9kB
8	Buffers: shared hit=1	
9	-> Seq Scan on language l	(cost=0.00..1.06 rows=6 width=88) (actual time=0.013..0.014 rows=6 loops=1)
10	Buffers: shared hit=1	
11	Planning:	
12	Buffers: shared hit=166 dirtied=1	
13	Planning Time:	1.564 ms
14	Execution Time:	0.660 ms
Total rows: 14 Query complete 00:00:00.063		
LF Ln 9, Col 1		

Hashed (Build phase): language l

Probed (Probe phase): film f

Memory Usage: 9kB

```
9  --Lab2
10 SET work_mem = '64kB';
11
12 EXPLAIN (ANALYZE , BUFFERS )
13 SELECT f.title , l.name as language
14 FROM film f
15 JOIN language l ON f.language_id = l.language_id;
16
```

Data Output	Messages	Notifications
Showing rows: 1 to 12		
Page No: 1 of 1		
SQL		
QUERY PLAN		
text		
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99) (actual time=0.046..0.455 rows=1000 loops=1)
2	Hash Cond	(f.language_id = l.language_id)
3	Buffers: shared hit=89	
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17) (actual time=0.022..0.201 rows=1000 loops=1)
5	Buffers: shared hit=88	
6	-> Hash	(cost=1.06..1.06 rows=6 width=88) (actual time=0.014..0.015 rows=6 loops=1)
7	Buckets: 1024 Batches: 1	Memory Usage: 9kB
8	Buffers: shared hit=1	
9	-> Seq Scan on language l	(cost=0.00..1.06 rows=6 width=88) (actual time=0.008..0.009 rows=6 loops=1)
10	Buffers: shared hit=1	
11	Planning Time:	0.269 ms
12	Execution Time:	0.509 ms

Result: The plan and execution behavior remain **unchanged** (it will still use a 1-batch in-memory Hash Join).

LAB3:

```
18 --Lab3
19 SET enable_hashjoin = OFF ;
20 SET enable_nestloop= OFF ;
21
22 EXPLAIN (ANALYZE , BUFFERS )
23 SELECT f.film_id , f.title , i.inventory_id
24 FROM film f
25 JOIN inventory i ON f.film_id = i.film_id
```

Data Output Messages Notifications

Showing rows: 1 to 17 Page No: 1 of 1

QUERY PLAN
text
1 Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=2.040..3.430 rows=4581 loops=1)
2 Merge Cond: (f.film_id = i.film_id)
3 Buffers: shared hit=111 read=1, temp read=10 written=12
4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.019..0.261 rows=1000 loops=1)
5 Buffers: shared hit=83 read=1
6 -> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=2.013..2.587 rows=4581 loops=1)
7 Buffers: shared hit=28, temp read=10 written=12
8 -> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=2.010..2.296 rows=4581 loops=1)
9 Sort Key: i.film_id
10 Sort Method: external merge Disk: 80kB
11 Buffers: shared hit=28, temp read=10 written=12
12 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.045..0.405 rows=4581 loops=1)
13 Buffers: shared hit=25
14 Planning:

Total rows: 17 Query complete 00:00:00.083 LF Ln 28, Col 1

Sort Node: Yes, an explicit **Sort node** is present for the inner relation

```
29 --Lab3
30 RESET ALL;
31 SET enable_hashjoin = OFF;
32 SET enable_nestloop = OFF;
33 EXPLAIN (ANALYZE, BUFFERS)
34 SELECT f.film_id, f.title, i.inventory_id
35 FROM film f
36 JOIN inventory i ON f.film_id = i.film_id;
```

Data Output Messages Notifications

Showing rows: 1 to 15 Page No: 1 of 1

QUERY PLAN
text
1 Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=1.151..2.352 rows=4581 loops=1)
2 Merge Cond: (f.film_id = i.film_id)
3 Buffers: shared hit=109
4 -> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.009..0.302 rows=1000 loops=1)
5 Buffers: shared hit=84
6 -> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=1.131..1.305 rows=4581 loops=1)
7 Sort Key: i.film_id
8 Sort Method: quicksort Memory: 300kB
9 Buffers: shared hit=25
10 -> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.014..0.505 rows=4581 loops=1)
11 Buffers: shared hit=25
12 Planning:
13 Buffers: shared hit=6
14 Planning Time: 0.263 ms

Total rows: 15 Query complete 00:00:00.072 LF Ln 29, Col 1

Does it switch back to Hash Join? Yes.

Why? With default settings re-enabled, the PostgreSQL query planner evaluates the estimated cost of all join strategies. For an equi-join between film (1,000 rows) and inventory (4,581 rows) without an ORDER BY clause, a **Hash Join** is chosen because building an in-memory hash table on the

smaller table (film) is computationally cheaper and avoids the overhead of sorting the larger inventory table.